

# The rkdiagram Package

Reed-Kellogg Sentence Diagrams for L<sup>A</sup>T<sub>E</sub>X

Version 1.0 - June 2026

© Karl McBurnett

kfm5634@gmail.com

## Introduction

---

The `rkdiagram` package typesets traditional Reed-Kellogg sentence diagrams in L<sup>A</sup>T<sub>E</sub>X. Each word is tagged with a command describing its grammatical role; the package computes the layout and draws everything with TikZ. No natural language parsing or processing occurs. The user should have a working knowledge of parts of speech for best results. A number of parts of speech are not included but may be in future versions. Reed-Kellogg sentence diagrams are a particularly American phenomenon which are now sparingly used in secondary schools and almost completely absent in linguistic studies. Because of this it seemed to me to be an interesting but harmless project - useful to a small number of educators but unlikely to be the lynch pin to someone else's endeavors.

**Features:** baselines, dividers, modifier diagonals, prepositional phrases, appositives, compound elements (including compound objects of prepositions), subordinate clauses, compound sentences, gerunds, and participles. Basic spacing and placement controls are available.

**Requirements:** Written using LuaL<sup>A</sup>T<sub>E</sub>X (but should work with any modern L<sup>A</sup>T<sub>E</sub>X engine) plus `tikz`, `xparse`, `etoolbox`, and `pgfopts`.

**AI Acknowledgement:** While I directed the development of this package, the majority of it and its documentation was written using Claude Sonnet 4.6 LLM as a learning exercise for myself in AI software development.

## Loading the Package

```
\usepackage[
  spacing      = 1.3cm,
  modDepth     = 1.0cm,
  modgap       = 0.25cm,
  cmpSep       = 0pt,
  subClauseDepth = 2.5cm,
  linewidth    = 0.5pt,
  font         = \rmfamily,
  color        = black,
]{rkdiagram}
```

The optional values above are the defaults. Override any option mid-document with `\rkset`:

```
\rkset{spacing=1.5cm, color=blue}
```

## Package options

| Option                      | Default    | Effect                                                                                              |
|-----------------------------|------------|-----------------------------------------------------------------------------------------------------|
| <code>spacing</code>        | 1.2cm      | Horizontal gap (padding) between baseline elements                                                  |
| <code>modDepth</code>       | 1.0cm      | Fixed horizontal step between plain adj/adv modifiers; minimum slant length for all modifiers/preps |
| <code>modgap</code>         | 0.25cm     | Extra vertical headspace between nested modifier levels                                             |
| <code>cmpSep</code>         | 0pt        | Minimum half-gap between compound platforms (0pt = auto)                                            |
| <code>subClauseDepth</code> | 2.5cm      | Vertical drop to a subordinate clause baseline                                                      |
| <code>gerundPost</code>     | 1.2cm      | Minimum post height for gerund T-platforms                                                          |
| <code>gerundPostH</code>    | 0pt        | Force exact post height, overriding adaptive calc (0pt = adaptive)                                  |
| <code>gerundShelfExt</code> | 0pt        | Extra width added to the right of the lower shelf and triangle foot                                 |
| <code>linewidth</code>      | 0.4pt      | Thickness of all diagram lines                                                                      |
| <code>font</code>           | (doc font) | Font applied to all diagram words                                                                   |
| <code>color</code>          | black      | Line and text colour                                                                                |
| <code>bridgeY</code>        | 0pt        | Pin compound-sentence bridge y (0pt = auto)                                                         |

## The rkdiagram Environment

Every diagram lives inside `rkdiagram`, which opens a *TikZ* picture:

```
\begin{rkdiagram}[spacing=1.5cm]    % optional per-diagram options
  \rkSubject{word}
  \rkVerb{word}
\end{rkdiagram}
```

Words are automatically centred on their baseline lines. The diagram sits on the text baseline and can appear inline or in its own paragraph.

## Baseline Elements

Baseline elements sit on the main horizontal line, placed left to right in the order written.

### Subject and Verb

A full vertical bar separates subject from verb. The bar's ascender (above the baseline) is full height; its descender (below the baseline) is half that height.

*“Dogs bark.”*

```
\begin{rkdiagram}
  \rkSubject{Dogs}
  \rkVerb{bark}
\end{rkdiagram}
```

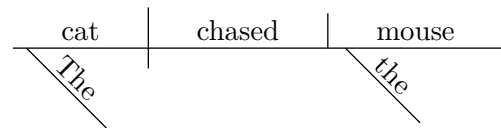


## Direct Object

`\rkDObj` places a short bar (above baseline only) after the verb.

*“The cat chased the mouse.”*

```
\begin{rkdiagram}
  \rkSubject[\rkAdj{The}]{cat}
  \rkVerb{chased}
  \rkDObj[\rkAdj{the}]{mouse}
\end{rkdiagram}
```

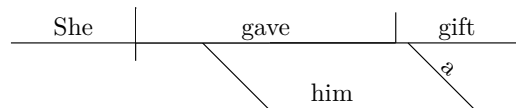


## Indirect Object

`\rkIObj` draws a diagonal platform below the verb midpoint.

*“She gave him a gift.”*

```
\begin{rkdiagram}
  \rkSubject{She}
  \rkVerb{gave}
  \rkIObj{him}
  \rkDObj[\rkAdj{a}]{gift}
\end{rkdiagram}
```

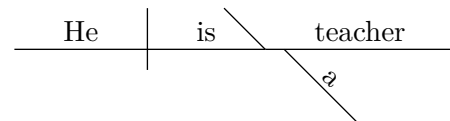


## Predicate Nominative and Predicate Adjective

After a linking verb, a back-diagonal divider (`\`) introduces a predicate nominative (`\rkPredNom`) or predicate adjective (`\rkPredAdj`). The diagonal slants at 45°, matching the modifier diagonals, and its bottom touches the baseline without descending below it.

*“He is a teacher.”*

```
\begin{rkdiagram}
  \rkSubject{He}
  \rkVerb{is}
  \rkPredNom[\rkAdj{a}]{teacher}
\end{rkdiagram}
```



## Modifiers

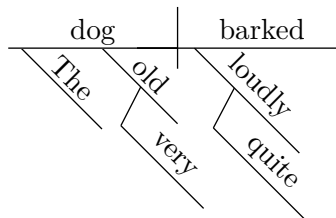
Modifiers hang below the word they modify on a 45° diagonal. Use `\rkAdj` for adjectives and articles, `\rkAdv` for adverbs. Pass them as the *optional first argument* of the word they modify, comma-separated:

```
\rkSubject[
  \rkAdj{The},
  \rkAdj{old}
]{dog}
```

A modifier takes its own optional argument for sub-modifiers, creating chains of diagonals:

*“The very old dog barked quite loudly.”*

```
\begin{rkdiagram}
  \rkSubject[
    \rkAdj{The},
    \rkAdj[\rkAdv{very}]{old}
  ]{dog}
  \rkVerb[
    \rkAdv[\rkAdv{quite}]{loudly}
  ]{barked}
\end{rkdiagram}
```



The `modgap` option adds extra vertical clearance between nested levels (default 0.25 cm), preventing crowding when modifier chains are deep.

**Modifier spacing:** plain adjectives and adverbs (no sub-modifiers) advance the sibling cursor by exactly `modDepth`, giving even horizontal spacing regardless of label width. Their slant lines are still sized to fit the word — long labels simply extend past the slant ends, matching hand-drawn convention. Modifiers that carry sub-modifiers (e.g. `\rkAdj[\rkAdv{very}]{old}`) use the word-fit formula so the slant is long enough to keep the sub-modifier connector clear of the next sibling. Use `modDepth` to control the global step between plain modifiers.

## Prepositional Phrases

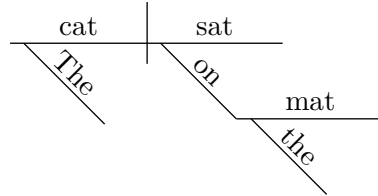
`\rkPrep` draws a diagonal with the preposition label, then a horizontal line for the object.

```
\rkPrep[mods-on-object]{preposition}{object}
```

Place it in the modifier list of the word it modifies:

*“The cat sat on the mat.”*

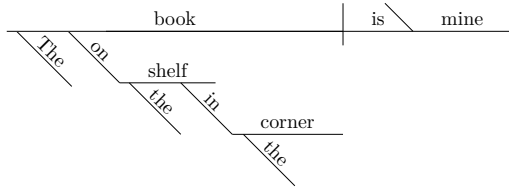
```
\begin{rkdiagram}
  \rkSubject[\rkAdj{The}]{cat}
  \rkVerb[
    \rkPrep[\rkAdj{the}]{on}{mat}
  ]{sat}
\end{rkdiagram}
```



Prepositional phrases nest freely — a prep modifying the object of another prep:

*“The book on the shelf in the corner is mine.”*

```
\begin{rkdiagram}
  \rkSubject[
    \rkAdj{The},
    \rkPrep[
      \rkAdj{the},
      \rkPrep[\rkAdj{the}]{in}{corner}
    ]{on}{shelf}
  ]{book}
  \rkVerb{is}
  \rkPredNom{mine}
\end{rkdiagram}
```



## Compound Structures

### Basic usage

`\rkCompound` joins two words or phrases with a conjunction. The full signature is:

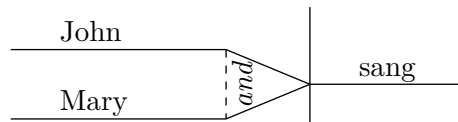
```
\rkCompound[conj][mods_a]{word_a}[mods_b]{word_b}
```

- `[conj]` — conjunction label (default: *and*)
- `[mods_a]` / `[mods_b]` — optional per-word modifier lists (same syntax as baseline commands)
- `{word_a}` / `{word_b}` — the two words or phrases

Each word sits on its own horizontal platform. The platforms are connected by convergence slants meeting at a point, a dashed vertical bar at the junction, and the conjunction written between the bar and the convergence point. Vertical separation scales with the conjunction width automatically.

*“John and Mary sang.”*

```
\begin{rkdiagram}
  \rkSubject{\rkCompound[and]{John}{Mary}}
  \rkVerb{sang}
\end{rkdiagram}
```

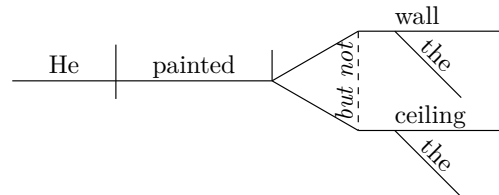


### Per-word modifiers

Modifiers on each compound word go in the optional modifier-list arguments:

*“He painted the wall but not the ceiling.”*

```
\begin{rkdiagram}
  \rkSubject{He}
  \rkVerb{painted}
  \rkDObj{
    \rkCompound[but not]
      [\rkAdj{the}]{wall}
      [\rkAdj{the}]{ceiling}
  }
\end{rkdiagram}
```



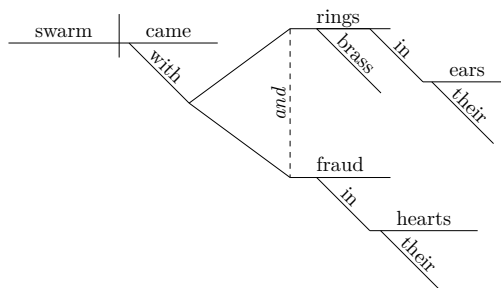
Modifiers placed in the list (e.g. `\rkAdj`, `\rkAdv`, `\rkDObj`) draw as proper diagonals on each word’s platform. Slot commands like `\rkDObj` inside a per-word modifier list attach a direct object to that compound word’s platform line.

## Compound object of a preposition

Pass `\rkCompound` as the object argument of `\rkPrep` to fork the preposition into two objects:

“...with brass rings in their ears and fraud in their hearts...”

```
\rkPrep{with}{  
  \rkCompound[and]  
  [  
    \rkAdj{brass},  
    \rkPrep[\rkAdj{their}]{in}{ears}  
  ]  
  {rings}  
  [\rkPrep[\rkAdj{their}]{in}{hearts}]  
  {fraud}  
}
```



Use `cmpSep` when modifiers on the compound words are deep and need extra vertical room between platforms.

## Compound Sentences

Use `\rkCompoundSent` *standalone* (not inside `rkdiagram`) to diagram two independent clauses in a single picture, connected by a dashed-line bridge with the coordinating conjunction. Each clause body must be wrapped in its own `rkdiagram` environment:

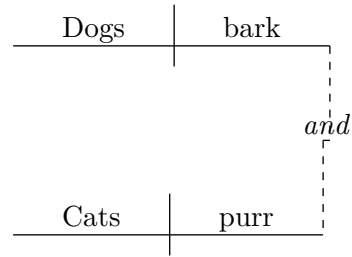
```
\rkCompoundSent[conj][sep][opts1]  
  {\begin{rkdiagram}body1\end{rkdiagram}}  
  [opts2]  
  {\begin{rkdiagram}body2\end{rkdiagram}}
```

- `[conj]` — conjunction (default: *and*)
- `[sep]` — vertical distance between the two baselines (default: 3 cm)
- `[opts1]` / `[opts2]` — optional per-clause `\rkset`-style options
- `{body1}` / `{body2}` — full `rkdiagram` environments for each clause

The connector drops a dashed vertical from each clause’s verb to a midpoint bridge, draws a short horizontal line there, and labels it with the conjunction.

*“Dogs bark and cats purr.”*

```
\rkCompoundSent[and][2.5cm]
{\begin{rkdiagram}
  \rkSubject{Dogs}
  \rkVerb{bark}
\end{rkdiagram}}
{\begin{rkdiagram}
  \rkSubject{Cats}
  \rkVerb{purr}
\end{rkdiagram}}
```

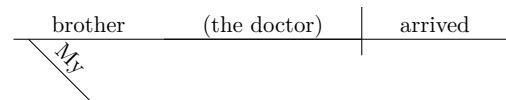


## Appositives

`\rkAppos` places an appositive in parentheses on the baseline, directly to the right of the word it renames. Modifier commands inside `\rkAppos` render inline.

*“My brother, the doctor, arrived.”*

```
\begin{rkdiagram}
  \rkSubject[
    \rkAdj{My},
    \rkAppos{\rkAdj{the} doctor}
  ]{brother}
  \rkVerb{arrived}
\end{rkdiagram}
```



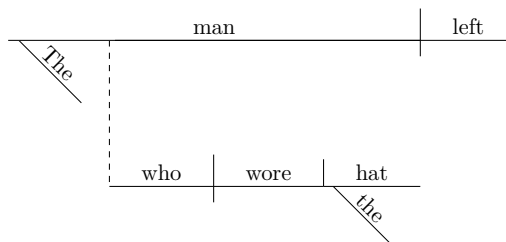
## Subordinate Clauses

`\rkSubClause` draws a complete sub-baseline below the main one, connected by a dashed vertical. It appears in the modifier list of the word it modifies. The optional argument is the relative or subordinating word placed first on the sub-baseline; it doubles as the grammatical subject, so do *not* add a separate `\rkSubject` call for it.

```
\rkSubClause[rel-word]{body}
```

“The man who wore the hat left.”

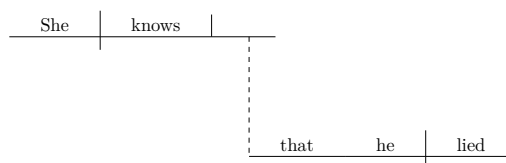
```
\begin{rkdiagram}
  \rkSubject[
    \rkAdj{The},
    \rkSubClause[who]{
      \rkVerb{wore}
      \rkDObj[\rkAdj{the}]{hat}
    }
  ]{man}
  \rkVerb{left}
\end{rkdiagram}
```



A noun clause as direct object — the sub-clause body starts directly with `\rkVerb`:

“She knows that he lied.”

```
\begin{rkdiagram}
  \rkSubject{She}
  \rkVerb{knows}
  \rkDObj{
    \rkSubClause[that]{
      \rkSubject{he}
      \rkVerb{lied}
    }
  }
\end{rkdiagram}
```



## Gerunds

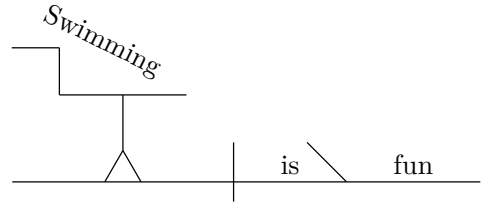
A *gerund* is a verb form used as a noun. It sits on a stepped T-platform: a full-width horizontal foot at the noun-slot baseline, two short angled legs rising to an apex, a vertical post, and a short word-platform at the top. Use `\rkGerund` as the *word argument* of any baseline command (`\rkSubject`, `\rkDObj`, `\rkPredNom`, etc.) or as the object of `\rkPrep`.

```
\rkGerund[mods]{word}
```

The optional `[mods]` argument holds modifiers of the gerund itself (adjectives, prepositional phrases, etc.). The package pre-scans the list to compute a post height tall enough to keep all modifier slants clear of the foot. Use `gerundPost` (default 1.2cm) to set a global minimum post height. Two additional `\rkset` controls allow per-gerund manual adjustment — see the subsections below.

*“Swimming is fun.”*

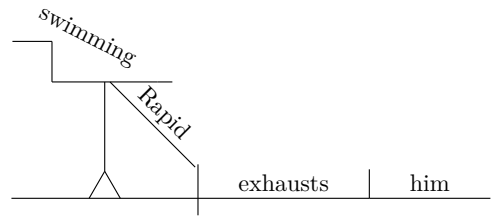
```
\begin{rkdiagram}
  \rkSubject{\rkGerund{Swimming}}
  \rkVerb{is}
  \rkPredAdj{fun}
\end{rkdiagram}
```



Gerund with its own modifiers — the post height adjusts automatically:

*“Rapid swimming exhausts him.”*

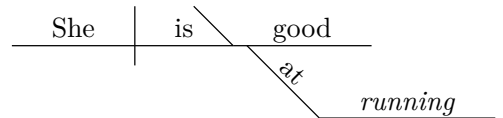
```
\begin{rkdiagram}
  \rkSubject{
    \rkGerund[\rkAdj{Rapid}]{swimming}
  }
  \rkVerb{exhausts}
  \rkDObj{him}
\end{rkdiagram}
```



Gerund as the object of a preposition:

*“She is good at running quickly.”*

```
\begin{rkdiagram}
  \rkSubject{She}
  \rkVerb{is}
  \rkPredAdj[
    \rkPrep{at}{
      \rkGerund[\rkAdv{quickly}]{running}
    }
  ]{good}
\end{rkdiagram}
```



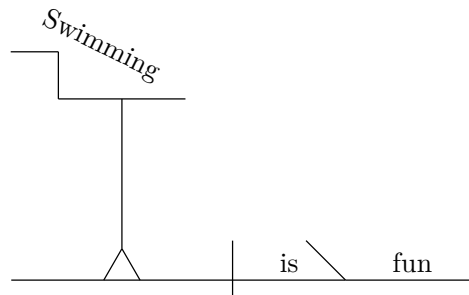
### gerundPostH — force exact post height

Normally the vertical bar height is chosen adaptively: at least `gerundPost`, and taller if the gerund’s own modifier slants require it. `gerundPostH` overrides both constraints and sets the post to exactly the given length. Set it back to `0pt` to restore adaptive behaviour for subsequent gerunds.

```
\rkset{gerundPostH=2.5cm} % force this one gerund's post to 2.5 cm
\rkSubject{\rkGerund{Swimming}}
\rkset{gerundPostH=0pt}   % restore adaptive mode
```

*“Swimming is fun. (forced post height)”*

```
\begin{rkdiagram}
  \rkset{gerundPostH=2.5cm}
  \rkSubject{\rkGerund{Swimming}}
  \rkVerb{is}
  \rkPredAdj{fun}
\end{rkdiagram}
\rkset{gerundPostH=0pt}
```



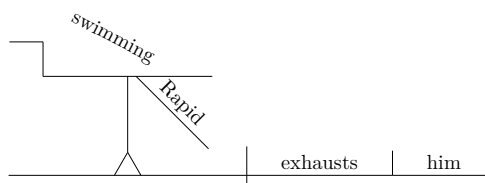
### gerundShelfExt — extend the lower shelf

By default the lower shelf spans exactly the gerund word’s width. When a gerund has wide or numerous modifiers, their slant lines may extend past the shelf’s right edge. **gerundShelfExt** adds extra length to the right of the lower shelf and the triangle foot, giving the modifiers more room.

```
\rkset{gerundShelfExt=1.5cm}
\rkSubject{\rkGerund[\rkAdj{Rapid}]{swimming}}
\rkset{gerundShelfExt=0pt}
```

*“Rapid swimming exhausts him. (extended shelf)”*

```
\begin{rkdiagram}
  \rkset{gerundShelfExt=1.5cm}
  \rkSubject{
    \rkGerund[\rkAdj{Rapid}]{swimming}
  }
  \rkVerb{exhausts}
  \rkDObj{him}
\end{rkdiagram}
\rkset{gerundShelfExt=0pt}
```



## Participles

A *participle* functions as an adjective modifying a noun. It is drawn with a hockey-stick shape: a 45° diagonal (word label above it), a smooth curve at the elbow, and a horizontal stub. Use **\rkParticiple** inside a modifier list — the same position as **\rkAdj**.

```
\rkParticiple[tail]{word}
```

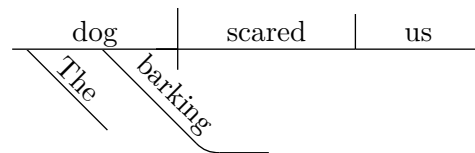
Without a tail argument the shape is simply the angled line with word label — enough for a bare participial adjective. With a tail, the horizontal stub serves as a mini-baseline for a participial direct object and/or modifiers of the participle:

```
% Participial object on the tail:
\rkPartObj[mods]{word}
```

```
% Adverb or prep phrase modifying the participle, also on the tail:
\rkAdv{quickly}
\rkPrep[obj-mods]{prep}{obj}
```

*“The barking dog scared us.”*

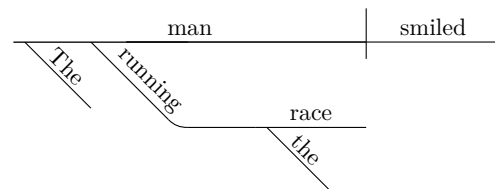
```
\begin{rkdiagram}
\rkSubject[
  \rkAdj{The},
  \rkParticiple{barking}
]{dog}
\rkVerb{scared}
\rkDObj{us}
\end{rkdiagram}
```



Participle with a participial object (`\rkPartObj`) on the tail:

*“The man running the race smiled.”*

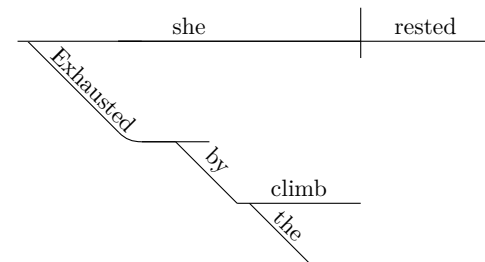
```
\begin{rkdiagram}
\rkSubject[
  \rkAdj{The},
  \rkParticiple[
    \rkPartObj[\rkAdj{the}]{race}
  ]{running}
]{man}
\rkVerb{smiled}
\end{rkdiagram}
```



Participle with a prepositional phrase on the tail (the traditional passive-participial form):

*“Exhausted by the climb, she rested.”*

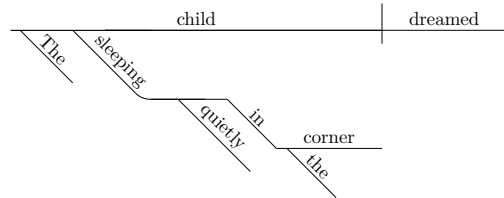
```
\begin{rkdiagram}
\rkSubject[
  \rkParticiple[
    \rkPrep[\rkAdj{the}]{by}{climb}
  ]{Exhausted}
]{she}
\rkVerb{rested}
\end{rkdiagram}
```



Multiple tail items (adverb + prepositional phrase) both branch from the horizontal stub:

*“The child sleeping quietly in the corner dreamed.”*

```
\begin{rkdiagram}
  \rkSubject[
    \rkAdj{The},
    \rkParticiple[
      \rkAdv{quietly},
      \rkPrep[\rkAdj{the}]{in}{corner}
    ]{sleeping}
  ]{child}
  \rkVerb{dreamed}
\end{rkdiagram}
```



## Infinitives

An infinitive (*to + verb*) has two distinct uses:

- **Noun function** — used as subject, direct object, or predicate nominative, just like a gerund. The diagram uses the same stepped T-pedestal as `\rkGerund`, but the word *to* is written on the vertical post. Use `\rkInfinitive` wherever you would use `\rkGerund`: as the word argument of `\rkSubject`, `\rkDObj`, etc.
- **Modifier function** — used as adjective or adverb. The diagram looks like a prepositional phrase: a diagonal line with *to* written on it, and the verb on a short horizontal at the bottom. That horizontal is a full mini-baseline: use `\rkInfObj` to place a direct object, and `\rkAdv/\rkPrep` for adverbial modifiers. Use `\rkInfinitive` inside any modifier list.

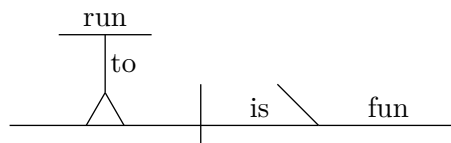
```
\rkInfinitive[arg]{verb}
```

The optional `arg` carries modifiers of the verb in the noun form; in the modifier form it is tail content (objects, adverbial modifiers) on the mini-baseline.

### Noun form — infinitive as subject or object

*“To run is fun.”*

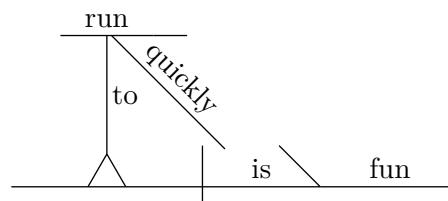
```
\begin{rkdiagram}
  \rkSubject{\rkInfinitive{run}}
  \rkVerb{is}
  \rkPredNom{fun}
\end{rkdiagram}
```



Infinitive noun with a modifier on the verb:

*“To run quickly is fun.”*

```
\begin{rkdiagram}
  \rkSubject{
    \rkInfinitive[\rkAdv{quickly}]{run}
  }
  \rkVerb{is}
  \rkPredNom{fun}
\end{rkdiagram}
```

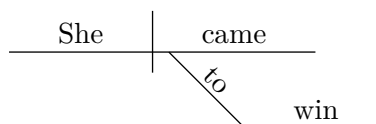


### Modifier form — infinitive as adjective or adverb

Without a tail the shape is a diagonal labeled *to* with the verb on a short horizontal — identical in form to a prepositional phrase.

*“She came to win.”*

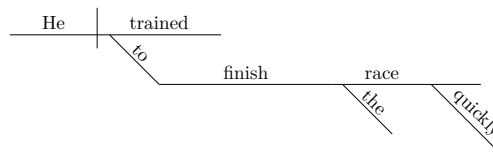
```
\begin{rkdiagram}
  \rkSubject{She}
  \rkVerb[\rkInfinitive{win}]{came}
\end{rkdiagram}
```



With tail content, use `\rkInfObj` for a direct object and `\rkAdv/\rkPrep` for adverbial modifiers of the infinitive verb:

*“He trained to finish the race quickly.”*

```
\begin{rkdiagram}
  \rkSubject{He}
  \rkVerb[
    \rkInfinitive[
      \rkInfObj[\rkAdj{the}]{race}
      \rkAdv{quickly}
    ]{finish}
  ]{trained}
\end{rkdiagram}
```



### Nominative Absolute

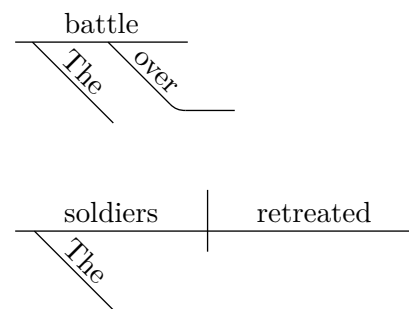
A nominative absolute is a noun-plus-participial-phrase that modifies the whole sentence rather than any single word. In a Reed-Kellogg diagram it floats above the main clause with no connector line.

```
\rkNomAbs[noun-mods]{noun}{ptcl-tail}{ptcl-word}
```

- **Call order:** `\rkNomAbs` can be called anywhere inside the diagram environment — it always floats above the main baseline at a fixed height.
- **Noun modifiers:** the optional `[noun-mods]` argument takes the same modifier syntax as `\rkSubject`’s modifier list (`\rkAdj`, etc.).
- **Participial tail:** the `{ptcl-tail}` argument is passed directly to `\rkParticiple` — use `\rkPartObj`, `\rkAdv`, or `\rkPrep` there; pass `{}` for an empty tail.
- **No participle:** leave `{ptcl-word}` empty for an absolute with an implied *being* (e.g. *his work done*).

“The battle over, the soldiers retreated.”

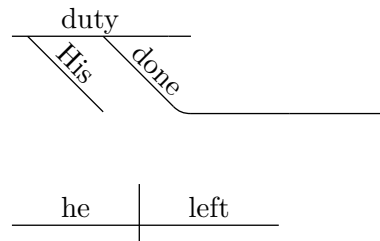
```
\begin{rkdiagram}
  \rkSubject[\rkAdj{The}]{soldiers}
  \rkVerb{retreated}
  \rkNomAbs[\rkAdj{The}]{battle}{}{over}
\end{rkdiagram}
```



Nominative absolute with a participial object on the tail:

“His duty done, he left.”

```
\begin{rkdiagram}
  \rkSubject{he}
  \rkVerb{left}
  \rkNomAbs[\rkAdj{His}]{duty}{
    \rkPartObj{
      }{done}
    }
\end{rkdiagram}
```



## Manual Spacing Controls

Three one-shot commands let you override automatic spacing for a single element; they reset to zero automatically after use.

```
\rkSlotWidth{len}  % min line width for the NEXT baseline slot
\rkSkip{len}       % gap in baseline BEFORE the next slot
\rkSlantLen{len}   % min diagonal length for the NEXT modifier/prep
```

### `\rkSlotWidth` — minimum word line length

Forces the next slot’s baseline line to be at least `len` wide, regardless of word width. Useful for aligning parallel diagrams or leaving annotation space.

*“Dogs bark. (4 cm subject line)”*

```
\begin{rkdiagram}
  \rkSlotWidth{4cm}
  \rkSubject{Dogs}
  \rkVerb{bark}
\end{rkdiagram}
```



### `\rkSkip` — gap before a slot

Inserts a gap of `len` in the baseline *before* the next slot. The gap is drawn as a continuation of the current baseline line, effectively pushing everything right from that point.

*“Dogs bark. (1.5 cm gap before verb)”*

```
\begin{rkdiagram}
  \rkSubject{Dogs}
  \rkSkip{1.5cm}
  \rkVerb{bark}
\end{rkdiagram}
```

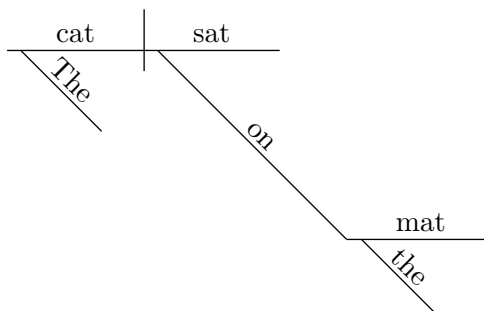


### `\rkSlantLen` — minimum diagonal length

Forces the next modifier or prepositional slant to be at least `len` long. Place it on the line immediately before the `\rkAdj`, `\rkAdv`, or `\rkPrep` call.

*“The cat sat on the mat. (longer prep slant)”*

```
\begin{rkdiagram}
  \rkSubject[\rkAdj{The}]{cat}
  \rkVerb[
    \rkSlantLen{2.5cm}
    \rkPrep[\rkAdj{the}]{on}{mat}
  ]{sat}
\end{rkdiagram}
```



All three commands work inside sub-clauses and on compound-word platforms as well as the main baseline.

### `\rkNomAbsHeight` — nominative absolute height

By default `\rkNomAbs` floats the absolute phrase at `subClauseDepth` above the main baseline. When the participle and its own modifiers are deep, they can crowd the main clause. Call `\rkNomAbsHeight` immediately before `\rkNomAbs` to override the height for that one phrase; it resets automatically.

```
\rkNomAbsHeight{4cm}
```

## Structural Depth Controls

The global options `modDepth`, `subClauseDepth`, and `cmpSep` control the vertical geometry of the diagram. Any of them can be changed per-diagram by passing options to the `rkdiagram` environment, or mid-document with `\rkset`:

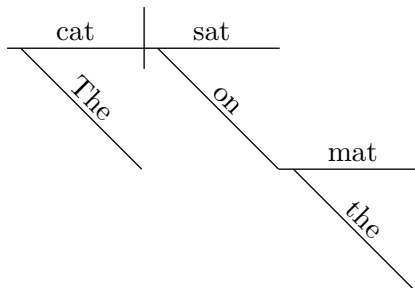
```
\begin{rkdiagram}[modDepth=1.5cm, subClauseDepth=3cm]
...
\end{rkdiagram}
```

### `modDepth` — modifier spacing and slant depth

Controls two related things simultaneously. For plain adjectives and adverbs (no sub-modifiers), it is the *fixed horizontal step* between siblings — all such modifiers are evenly spaced regardless of label width, with their slant lines sized to fit the word. For modifiers with sub-modifiers, it is the *minimum* slant length. Increase it to widen spacing and push labels further from the baseline; decrease it to tighten compact diagrams.

“*Deeper slants (modDepth=1.6cm)*”

```
\begin{rkdiagram}[modDepth=1.6cm]
\rkSubject[\rkAdj{The}]{cat}
\rkVerb[\rkPrep[\rkAdj{the}]{on}{mat}]{sat}
\end{rkdiagram}
```

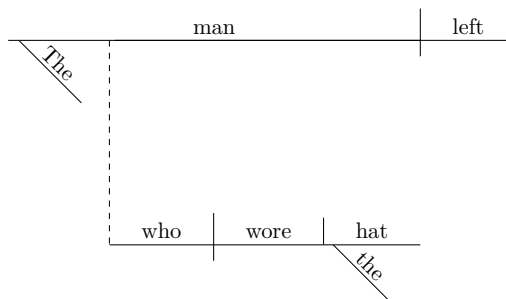


### `subClauseDepth` — vertical drop to sub-baseline

Controls how far below the main baseline a subordinate clause is drawn. Increase it when deep modifier chains on the main clause would overlap the sub-clause.

“Deeper sub-clause (*subClauseDepth=3.5cm*)”

```
\begin{rkdiagram}[subClauseDepth=3.5cm]
  \rkSubject[
    \rkAdj{The},
    \rkSubClause[who]{
      \rkVerb{wore}
      \rkDObj[\rkAdj{the}]{hat}
    }
  ]{man}
  \rkVerb{left}
\end{rkdiagram}
```

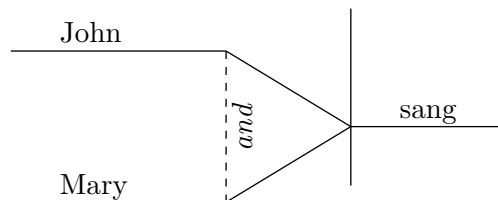


### cmpSep — compound platform separation

Sets the minimum vertical half-gap between the two platforms of a compound element. The default `Opt` lets the package scale the gap automatically from the conjunction width. Set an explicit value when modifier chains on the compound words need extra vertical clearance.

“Extra separation (*cmpSep=1.0cm*)”

```
\begin{rkdiagram}[cmpSep=1.0cm]
  \rkSubject{
    \rkCompound[and]{John}{Mary}
  }
  \rkVerb{sang}
\end{rkdiagram}
```



### bridgeY — connector bridge height (compound sentences)

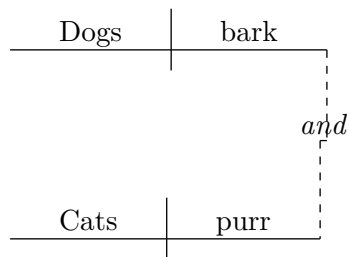
By default `\rkCompoundSent` places the horizontal bridge halfway between the two baselines, automatically lowering it if clause 1 modifiers extend into that region. Use `bridgeY` (a negative length, measured from clause 1’s baseline) to pin the bridge at an exact position:

```
\rkCompoundSent[and][3cm][bridgeY=-1.8cm]
{
  \begin{rkdiagram}
    \rkSubject{Dogs}
    \rkVerb{bark}
  \end{rkdiagram}
}
{
  \begin{rkdiagram}
    \rkSubject{Cats}
    \rkVerb{purr}
  \end{rkdiagram}
}
```

Pass `bridgeY` in the first per-clause options argument. Setting it to `Opt` restores automatic placement.

*“Bridge pinned at  $-1.2\text{ cm}$ ”*

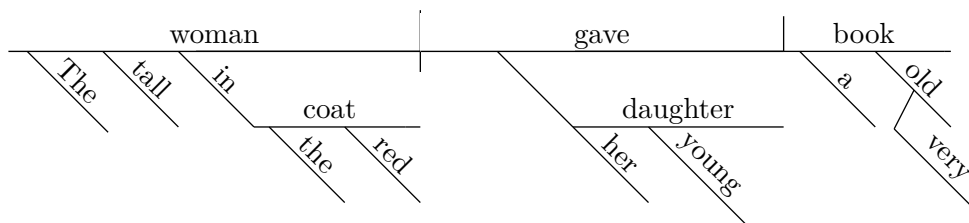
```
\rkCompoundSent[and][2.5cm]
[bridgeY=-1.2cm]
{\begin{rkdiagram}
  \rkSubject{Dogs} \rkVerb{bark}
\end{rkdiagram}}
{\begin{rkdiagram}
  \rkSubject{Cats} \rkVerb{purr}
\end{rkdiagram}}
```



## Complete Example

*“The tall woman in the red coat gave her young daughter a very old book.”*

```
\begin{rkdiagram}
  \rkSubject[
    \rkAdj{The},
    \rkAdj{tall},
    \rkPrep[
      \rkAdj{the},
      \rkAdj{red}
    ]{in}{coat}
  ]{woman}
  \rkVerb{gave}
  \rkIObj[
    \rkAdj{her},
    \rkAdj{young}
  ]{daughter}
  \rkDObj[
    \rkAdj{a},
    \rkAdj[\rkAdv{very}]{old}
  ]{book}
\end{rkdiagram}
```



## Command Reference

### Baseline commands

All baseline commands share the signature `\rkXxx[modifier-list]{word(s)}`. The modifier list is a comma-separated sequence of modifier, prep, appositive, or sub-clause calls.

| Command                             | Role                                             |
|-------------------------------------|--------------------------------------------------|
| <code>\rkSubject[mods]{word}</code> | Subject; left of the full vertical divider       |
| <code>\rkVerb[mods]{word}</code>    | Verb; right of the full vertical divider         |
| <code>\rkDObj[mods]{word}</code>    | Direct object; right of the short vertical bar   |
| <code>\rkIObj[mods]{word}</code>    | Indirect object; diagonal platform below verb    |
| <code>\rkPredNom[mods]{word}</code> | Predicate nominative; right of the back-diagonal |
| <code>\rkPredAdj[mods]{word}</code> | Predicate adjective; right of the back-diagonal  |

### Modifier and structural commands

| Command                                                       | Role                                                                                                                                                                        |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>\rkAdj[mods]{word}</code>                               | Adjective/article; diagonal below modified noun                                                                                                                             |
| <code>\rkAdv[mods]{word}</code>                               | Adverb; diagonal below modified verb/adj/adv                                                                                                                                |
| <code>\rkPrep[obj-mods]{prep}{obj}</code>                     | Prep phrase; diagonal + horizontal object line                                                                                                                              |
| <code>\rkAppos{content}</code>                                | Appositive; parenthesized on the baseline                                                                                                                                   |
| <code>\rkSubClause[rel]{body}</code>                          | Subordinate clause; dashed connector to sub-baseline                                                                                                                        |
| <code>\rkCompound[conj][mA]{wA}[mB]{wB}</code>                | Compound element; forked platforms with conjunction                                                                                                                         |
| <code>\rkGerund[mods]{word}</code>                            | Gerund; stepped T-platform (used as word argument of any baseline command or <code>\rkPrep</code> object)                                                                   |
| <code>\rkInfinitive[arg]{verb}</code>                         | Infinitive; noun form (pedestal + <i>to</i> on post) when used as word argument; modifier form (diagonal labeled <i>to</i> + verb on horizontal) when used in modifier list |
| <code>\rkInfObj[mods]{word}</code>                            | Direct object on the infinitive mini-baseline; only inside <code>\rkInfinitive</code> 's tail argument                                                                      |
| <code>\rkParticiple[tail]{word}</code>                        | Participle; hockey-stick modifier of a noun (used inside modifier lists like <code>\rkAdj</code> )                                                                          |
| <code>\rkPartObj[mods]{word}</code>                           | Participial object on the tail; only inside <code>\rkParticiple</code> 's tail argument                                                                                     |
| <code>\rkNomAbs[noun-mods]{noun}{ptcl-tail}{ptcl-word}</code> | Nominative absolute; floating phrase above the main clause, no connector                                                                                                    |

### Compound sentence

| Command                                                             | Role                                                                                                           |
|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>\rkCompoundSent[conj][sep][opts1]{body1}[opts2]{body2}</code> | Compound sentence (standalone; each body wraps its own <code>\rkdiagram</code> ), connected by a dashed bridge |

## Manual spacing (one-shot)

| Command                           | Effect                                             |
|-----------------------------------|----------------------------------------------------|
| <code>\rkSlotWidth{len}</code>    | Minimum line width for the next baseline slot      |
| <code>\rkSkip{len}</code>         | Gap in baseline before the next slot               |
| <code>\rkSlantLen{len}</code>     | Minimum diagonal length for the next modifier/prep |
| <code>\rkNomAbsHeight{len}</code> | Height above main baseline for the next            |
|                                   | <code>\rkNomAbs</code>                             |

## Structural depth options (per-diagram or global)

These are standard package options (see §2) but can also be passed to any `rkdiagram` environment or to `\rkset` mid-document:

| Option                          | Effect                                                                                               |
|---------------------------------|------------------------------------------------------------------------------------------------------|
| <code>modDepth=len</code>       | Fixed step between plain adj/adv siblings; minimum slant length for sub-modified modifiers and preps |
| <code>subClauseDepth=len</code> | Vertical drop from main baseline to sub-clause baseline                                              |
| <code>cmpSep=len</code>         | Minimum half-gap between compound platforms ( <code>Opt = auto</code> )                              |
| <code>bridgeY=neg-len</code>    | Pin compound-sentence bridge at a fixed y below clause 1 ( <code>Opt = auto</code> )                 |
| <code>gerundPost=len</code>     | Global minimum post height for gerund vertical bars                                                  |
| <code>gerundPostH=len</code>    | Force exact post height for the next gerund ( <code>Opt = adaptive</code> )                          |
| <code>gerundShelfExt=len</code> | Extend gerund lower shelf and foot to the right ( <code>Opt = auto</code> )                          |

## Global configuration

| Command                          | Effect                                   |
|----------------------------------|------------------------------------------|
| <code>\rkset{key=val,...}</code> | Override any package option mid-document |

## rkdiagram Macro Reference

This section describes every public macro provided by the `rkdiagram` package. Internal helpers (names beginning with `\rk@`) are not listed.

### Environment

`\begin{rkdiagram}[opts] ... \end{rkdiagram}` The main diagramming environment. Opens a `tikzpicture` and initialises all internal registers. The optional *opts* argument accepts any `rkdiagram` key–value pair (same as `\rkset`), allowing per-diagram overrides of global defaults.

All baseline commands, modifier commands, and structural commands must appear inside this environment (or inside a body argument of `\rkCompoundSent`).

### Compound sentence

`\rkCompoundSent[conj][sep][opts1]{body1}[opts2]{body2}` Draws two independent clauses in a single `tikzpicture` connected by a dashed bridge bearing the coordinating conjunction.

| Argument     | Meaning                                                                       |
|--------------|-------------------------------------------------------------------------------|
| <i>conj</i>  | Conjunction label rendered in italics above the bridge (default: <i>and</i> ) |
| <i>sep</i>   | Vertical distance between the two baselines (default: 3cm)                    |
| <i>opts1</i> | Per-diagram <code>rkdiagram</code> options applied to clause 1                |
| <i>body1</i> | Complete first clause, wrapped in its own <code>rkdiagram</code> environment  |
| <i>opts2</i> | Per-diagram options applied to clause 2                                       |
| <i>body2</i> | Complete second clause, wrapped in its own <code>rkdiagram</code> environment |

The bridge y-position is computed automatically (midway between the baselines, lowered if clause-1 modifiers extend into that region) but can be overridden with the `bridgeY` key.

### Baseline commands

All six baseline commands share the optional/mandatory signature `\rkXxx[mods]{word}`. The *mods* argument is a comma-separated list of modifier, prep-phrase, appositive, or subordinate-clause calls.

`\rkSubject[mods]{word}` Places the subject word on the main baseline and draws a full vertical divider to its right. *word* may itself be `\rkGerund`, `\rkInfinitive`, or `\rkCompound`. *mods* attaches adjective/article slants below the subject's platform.

`\rkVerb[mods]{word}` Places the verb word immediately to the right of the full vertical divider. *word* may be `\rkCompound` for a compound verb. *mods* attaches adverb and prepositional-phrase slants below the verb's platform.

`\rkDObj[mods]{word}` Draws a short vertical bar and places the direct-object word to its right. *word* may be `\rkGerund`, `\rkInfinitive`, or `\rkCompound`. *mods* attaches adjective slants below the object's platform.

`\rkPredNom[mods]{word}` Draws a back-diagonal divider (`\`) and places the predicate nominative to its right. *word* may be `\rkCompound`. *mods* attaches adjective slants below the platform.

`\rkPredAdj[mods]{word}` Identical in structure to `\rkPredNom`; use this command when the predicate word is an adjective to distinguish it semantically.

`\rkIObj[mods]{word}` Draws the indirect-object platform: a short diagonal drops below the midpoint of the verb slot, then a horizontal line extends to the right bearing the object word. Does not advance the main baseline cursor. *mods* attaches slants below the indirect-object platform.

## Modifier commands

`\rkAdj[submods]{word}` Draws a 45° slant descending to the right from the current attachment cursor, with *word* labelled above the slant. Used for adjectives, articles, and any word that modifies a noun. The optional *submods* list attaches further modifiers (adverbs modifying the adjective, etc.) branching off the parent slant at a lower level.

`\rkAdv[submods]{word}` Identical in geometry to `\rkAdj`; use this command when the modifying word is an adverb. Attaches to verbs, adjectives, or other adverbs via the same 45° slant. *submods* may contain nested `\rkAdv` or `\rkPrep` calls.

`\rkPrep[obj-mods]{prep}{obj}` Draws a 45° slant labelled with *prep*, then a horizontal line bearing *obj*. *obj-mods* attaches adjective/article slants below the object's horizontal. *obj* may be `\rkCompound` for a compound object of the preposition.

## Compound structures

`\rkCompound[conj][mA]{wA}[mB]{wB}` When used as the *word* argument of any baseline command or as the object of `\rkPrep`, draws a forked compound platform: two parallel horizontal rails connected to the slot's attachment point by convergence slants, with a dashed vertical bar and the conjunction label between them.

| Argument    | Meaning                                                           |
|-------------|-------------------------------------------------------------------|
| <i>conj</i> | Conjunction rendered in italics on the bar (default: <i>and</i> ) |
| <i>mA</i>   | Modifier list for the upper word <i>wA</i>                        |
| <i>wA</i>   | First (upper) compound element                                    |
| <i>mB</i>   | Modifier list for the lower word <i>wB</i>                        |
| <i>wB</i>   | Second (lower) compound element                                   |

Compound subjects fork to the right (convergence on the right); compound verbs, objects, and predicate words fork to the left (convergence on the left). The direction is determined automatically by which baseline command contains the `\rkCompound` call.

## Appositives

`\rkAppos{content}` Places *content* in parentheses immediately to the right of the current slot on the same baseline. Intended for use inside the *mods* argument of `\rkSubject`, `\rkDObj`, etc. Adjective and adverb modifiers nested inside *content* are rendered inline (not as slants) so the appositive reads as ordinary text.

## Subordinate clauses

`\rkSubClause[rel]{body}` Draws a dashed vertical connector from the current modifier attachment point down to a sub-baseline, then executes *body* as a complete miniature clause at that level. The optional *rel* argument places a relative pronoun or subordinating conjunction word on the sub-baseline before the body. *body* may contain `\rkSubject`, `\rkVerb`, `\rkDObj`, etc.

## Gerunds

`\rkGerund[mods]{word}` Used as the *word* argument of any baseline command or as the object of `\rkPrep`. Draws the standard gerund pedestal: a full-width horizontal foot at the slot's

baseline level, two legs forming an equilateral triangle, a vertical post rising from the apex, and a short word platform at the top. The post height is computed adaptively so that all modifier slants in *mods* clear the foot.

## Infinitives

`\rkInfinitive[arg]{verb}` Has two distinct forms depending on context.

**Noun form** (used as the *word* argument of `\rkSubject`, `\rkDObj`, etc.): draws the same stepped pedestal as `\rkGerund` but labels the vertical post with *to*. *arg* is the modifier list for the infinitive verb on the platform.

**Modifier form** (used inside a *mods* list): draws a 45° slant labelled *to*, then a horizontal mini-baseline bearing *verb*. *arg* is the tail content for the mini-baseline: use `\rkInfObj` to attach a direct object and `\rkAdv/\rkPrep` for further modifiers of the infinitive verb.

`\rkInfObj[mods]{word}` Places a direct object on the infinitive mini-baseline. Valid only inside the *arg* (tail-content) argument of `\rkInfinitive` when used in modifier form. *mods* attaches adjective slants below the object's platform.

## Participles

`\rkParticiple[tail]{word}` A modifier command (used inside modifier lists like `\rkAdj`) that draws the hockey-stick shape for a participial phrase: a 45° slant with *word* labelled above it, a smooth curve at the elbow, and a short horizontal stub. The optional *tail* argument executes on the stub baseline; use `\rkPartObj` to add a participial object and `\rkAdv/\rkPrep` for modifiers of the participle.

`\rkPartObj[mods]{word}` Places a direct object on the participial tail. Valid only inside the *tail* argument of `\rkParticiple`. *mods* attaches adjective slants below the object's platform.

## Nominative absolute

`\rkNomAbs[noun-mods]{noun}{ptcl-tail}{ptcl-word}` Draws a nominative absolute phrase floating above the main clause with no connector.

| Argument         | Meaning                                                                                                                          |
|------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>noun-mods</i> | Modifier list for the noun (adjectives, articles, etc.)                                                                          |
| <i>noun</i>      | The noun of the absolute phrase                                                                                                  |
| <i>ptcl-tail</i> | Tail content for the participle ( <code>\rkPartObj</code> , <code>\rkAdv</code> , <code>\rkPrep</code> , etc.); pass {} for none |
| <i>ptcl-word</i> | The participial word; pass {} to omit the participle entirely                                                                    |

By default the phrase sits `subClauseDepth` above the main baseline. Precede the call with `\rkNomAbsHeight{len}` to override the height for that one phrase.

## Spacing override commands

Each command below is *one-shot*: it takes effect for the very next applicable operation, then resets automatically to its default.

| Command                                  | Effect                                                                                                                                                                |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>\rkSlotWidth{<i>len</i>}</code>    | Sets a minimum width for the next baseline slot's horizontal line. Useful when a short word would otherwise produce an unusually narrow platform.                     |
| <code>\rkSkip{<i>len</i>}</code>         | Inserts a gap of <i>len</i> in the baseline before the next slot is drawn. Useful for pausing the baseline to accommodate a wide structure beneath it.                |
| <code>\rkSlantLen{<i>len</i>}</code>     | Sets a minimum length for the next modifier or prepositional-phrase slant. Useful when a very short word would produce a slant too small to read.                     |
| <code>\rkNomAbsHeight{<i>len</i>}</code> | Overrides the height above the main baseline for the very next <code>\rkNomAbs</code> call. When <code>Opt</code> (the default), <code>subClauseDepth</code> is used. |

### Global configuration

`\rkset{key=val,...}` Applies any package key–value option mid-document, overriding the defaults set at load time. The same keys accepted as package options (see §2 and the Structural Depth Controls chapter) are all valid here: `font`, `color`, `linewidth`, `spacing`, `modDepth`, `modgap`, `cmpSep`, `subClauseDepth`, `gerundPost`, `bridgeY`. Changes persist from the point of the call onwards; scope them inside a  $\text{\LaTeX}$  group to limit their effect to a single diagram.

This work is licensed under a Creative Commons “Attribution-NonCommercial-ShareAlike 4.0 International” license.

